

Laboratorio 3 — Protocolos de Enrutamiento

Informe técnico de implementación

Curso	CC3067 Redes — Sección 20	Fecha	13 de agosto de 2026
Autores	Diego López y Javier López		
Repositorio	Código entregado con este informe: lab3-redes.		

1. Descripción de la práctica

Se implementó una red simulada de routers con protocolo *Link State* sobre TCP. Cada router descubre el estado de sus vecinos, difunde su información de enlaces mediante LSA, reconstruye el grafo de red y calcula rutas de menor costo con Dijkstra. La tabla resultante se persiste como CSV y alimenta el plano de datos.

El plano de datos recibe tramas codificadas con Hamming(7,4), corrige un error de bit por bloque cuando corresponde, recupera el mensaje JSON y lo reenvía al siguiente salto indicado por la tabla. La práctica incluye una aplicación ATM-Banco: ambos son endpoints, no routers, y se conectan a sus respectivos gateways mediante sockets.

2. Arquitectura y protocolo

Plano de control: HELLO → vecinos vivos → LSA/flooding → grafo → Dijkstra → CSV
Plano de datos: frame Hamming → corrección → JSON/destino → tabla CSV → siguiente salto

- **Transporte:** TCP; una conexión corta por mensaje y una línea terminada en salto de línea.
- **Puertos:** datos en el puerto configurado y control en `puerto_datos + 1000`.
- **Configuración:** `config/nodos.json`, `config/topologia.json` y `config/endpoints.json`.
- **Concurrencia:** servidores de control y datos, envío/monitoreo HELLO y refresco LSA se ejecutan en hilos daemon.

3. Algoritmos utilizados e implementación

3.1 Descubrimiento y Link State

HelloManager envía `{"type":"HELLO","from":id}` cada 2 s a cada vecino configurado. Un vecino pasa a UP al recibir su HELLO; si no se recibe durante 6 s, pasa a DOWN. Ambos cambios generan un LSA propio, por lo que la topología anunciada representa solamente los enlaces vivos.

LSAManager construye LSAs con `origin`, `seq`, `ttl`, `links` y `from`. La secuencia usa milisegundos y se fuerza estrictamente creciente para tolerar reinicios. Al recibir un LSA nuevo se actualiza el grafo y se reenvía a los vecinos, excepto al último salto; LSAs duplicados o antiguos se descartan y el TTL limita la propagación.

3.2 Grafo y ruta de menor costo

NetworkGraph reemplaza la lista de adyacencias de cada origen con el LSA más reciente. Sobre una instantánea del grafo, `compute_routes` implementa Dijkstra con una cola de prioridad (heapq). Para cada destino alcanzable devuelve (`costo`, `siguiente_salto`); el origen y nodos no alcanzables no se escriben como rutas.

La tabla se guarda en `data/nodo_<id>_tabla_enrutamiento.csv` con las columnas `destino`, `siguiente_salto`, `ip`, `puerto`, `costo`. El plano de datos la recarga después de cada recálculo.

3.3 Detección y corrección de errores

El texto JSON se codifica a UTF-8 y luego a bits. `encode_frame` divide la secuencia en bloques de 4 bits, añade relleno si es necesario y aplica Hamming(7,4); un encabezado de 2 bits conserva la cantidad de relleno. `decode_frame` revierte el proceso y `decode` calcula el síndrome para corregir un único bit por bloque. Si una trama no puede reconstruirse como texto/JSON válido, se descarta.

3.4 Reenvío y endpoints

ForwardingLayer obtiene el destino del mensaje, resuelve su gateway si es un endpoint y consulta el CSV para identificar IP y puerto del siguiente salto. Re-serializa el mismo mensaje y aplica Hamming antes de enviarlo. Cuando el gateway es local, entrega directamente al ATM o Banco. La aplicación bancaria implementa autenticación, retiro y cierre de sesión; ATM1 se conecta a A y BANC01 a D según la configuración incluida.

4. Resultados

Se ejecutó una validación local determinista de los módulos sin dependencias externas. Todos los casos finalizaron correctamente.

Prueba	Resultado observado	Evidencia en el repositorio
Hamming(7,4)	5 vectores de datos se recuperaron sin cambio; se corrigieron los 7 posibles errores de un bit de un bloque.	tests/test_hamming.py
Tramas UTF-8	5 textos, incluido ñ€ y una cadena de 50 caracteres, realizaron recorrido completo de codificación/decodificación.	tests/test_hamming.py
Dijkstra	Para A→C se eligió A→B→C, costo 3, en lugar del enlace directo de costo 4.	tests/test_routing.py
LSA	Las secuencias propias son crecientes; un LSA repetido o antiguo no se procesa nuevamente.	tests/test_lsa_flooding.py
Dos routers locales	Los logs muestran enlace A–B UP y tablas CSV recíprocas de costo 1.	logs/A.out, logs/B.out, CSV A/B

La ejecución se hizo con un script equivalente a los casos de prueba, porque el entorno de revisión no tenía `pytest` instalado. La suite incluida contiene 10 pruebas: 3 de Hamming/enmarcado, 4 de LSA y 3 de rutas.

5. Discusión

La separación entre plano de control y plano de datos mantiene clara la responsabilidad de cada componente: HELLO/LSA producen conocimiento de red; Dijkstra lo convierte en rutas; el reenvío solamente consume esas rutas. Esta organización evita duplicar lógica y permite sustituir la topología de prueba por la topología Tailscale desde archivos JSON.

El flooding evita ciclos innecesarios mediante `seq`, exclusión del vecino emisor y TTL. El refresco periódico de 15 s reduce el riesgo de que un router recién iniciado no reciba información previa. Hamming(7,4) cumple el requisito de corrección de un bit por bloque, pero no garantiza corregir errores múltiples; una trama inválida se descarta en vez de reenviarse corrupta.

Como limitaciones prácticas, la topología incluida es un placeholder local y las rutas solo reflejan LSAs recibidos. La validación final debe comprobar que las tres parejas usan exactamente el mismo formato de control/datos, IDs basados en IP Tailscale y el mismo offset de puerto.

6. Conclusiones y comentarios

1. La implementación cumple el flujo requerido: descubrimiento de vecinos, LSA con flooding, reconstrucción de grafo, Dijkstra, CSV y reenvío por sockets.
2. La tabla de ruteo permite escoger el siguiente salto óptimo; la prueba A→C confirma que se prefiere una ruta multi-salto menos costosa frente a un enlace directo más caro.
3. Hamming(7,4) se integra en cada salto de datos, preservando el JSON y corrigiendo un error de bit por bloque.

4. Antes de la entrega/demostración se deben reemplazar los IDs locales por IPs Tailscale, acordar el protocolo con las otras parejas y registrar una prueba extremo a extremo ATM–Banco a través de la topología de seis nodos.

7. Referencias

1. Universidad del Valle de Guatemala. *Lab 3 — Algoritmos de Enrutamiento — 2026*, CC3067 Redes.
2. Repositorio del grupo lab3–redes: código fuente, configuraciones, pruebas y logs de esta entrega.
3. Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik, 1, 269–271.
4. Hamming, R. W. (1950). *Error detecting and error correcting codes*. Bell System Technical Journal, 29(2), 147–160.